



HMS Backend – Full File Structure

This document provides a complete and modular file structure for the backend of the Hotel Management System, built using **Spring Boot**, **JPA**, and **PostgreSQL**.

Root Project Structure

```
hms-backend/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/hms/
│   │   │   │   ├── HmsBackendApplication.java
│   │   │   │   ├── config/
│   │   │   │   ├── auth/
│   │   │   │   ├── rooms/
│   │   │   │   ├── bookings/
│   │   │   │   ├── guests/
│   │   │   │   ├── billing/
│   │   │   │   ├── finance/
│   │   │   │   ├── housekeeping/
│   │   │   │   ├── staff/
│   │   │   │   ├── feedback/
│   │   │   │   ├── inventory/
│   │   │   │   ├── iot/
│   │   │   │   ├── reports/
│   │   │   │   ├── settings/
│   │   │   │   └── common/
│   │   ├── resources/
│   │   │   ├── application.properties
│   │   │   ├── messages.properties
│   │   │   └── templates/
│   │   │       ├── invoice_template.html
│   │   │       └── report_template.html
│   └── pom.xml
```

Detailed Package Responsibilities

☒ config/

- WebConfig.java
- SecurityConfig.java
- CorsConfig.java
- SwaggerConfig.java

☒ auth/

- AuthController.java, AuthService.java
- JwtUtil.java, JwtFilter.java
- DTOs: LoginRequest, RegisterRequest, LoginResponse
- Entity: User.java, Role.java

☒ rooms/

- RoomController.java, RoomService.java, RoomRepository.java
- Entity: Room.java
- DTOs: CreateRoomRequest, RoomDTO

☒ bookings/

- BookingController.java, BookingService.java, BookingRepository.java
- Entity: Booking.java

☒ guests/

- GuestController.java, GuestService.java
- Entity: Guest.java

☒ billing/

- BillingController.java, InvoiceService.java
- Entity: Invoice.java

☒ **finance/**

- FinanceController.java, ExpenseService.java
- Entities: Expense.java, Revenue.java

☒ **housekeeping/**

- HousekeepingController.java, HousekeepingService.java
- Entity: HousekeepingTask.java

☒ **staff/**

- StaffController.java, StaffService.java
- Entity: Staff.java

☒ **feedback/**

- FeedbackController.java, FeedbackService.java
- Entity: Feedback.java

☒ **inventory/**

- InventoryController.java, InventoryService.java
- Entity: InventoryItem.java

☒ **iot/**

- DeviceController.java, SmartRoomService.java
- Entity: DeviceRegistry.java

☒ **reports/**

- ReportController.java
- Services: PdfExportService, ExcelExportService

☑ settings/

- SettingsController.java, SettingsService.java
- Entity: GlobalSetting.java

☑ common/

- ApiResponse.java, BaseController.java
- exception/: GlobalExceptionHandler, ResourceNotFoundException
- util/: EmailUtil, DateUtil, FileUtil

🔧 Technologies Used

Component	Technology
Framework	Spring Boot
Database	PostgreSQL
Security	JWT, Spring Security
Export	Apache POI, iText PDF
Docs	SpringDoc OpenAPI

💻 2. JavaFX Desktop App (hms-desktop-ui/)

```
bash
CopyEdit
hms-desktop-ui/
├── src/
│   ├── main/
│   │   ├── java/com/hms/desktop/
│   │   │   ├── Main.java           # JavaFX launcher
│   │   │   ├── controllers/       # HomeController,
│   │   │   │   BookingController, etc.
│   │   │   ├── services/          # API connectors
│   │   │   └── models/            # POJOs matching backend
└── DTOs
```

```

|   |   |   └─ utils/           # QR scanner, file reader
|   |   └─ resources/
|   |       └─ fxml/           # FXML layouts
|   |       └─ css/            # Stylesheets
|   |       └─ icons/          # UI icons
└─ build.gradle or pom.xml

```

3. JSP Website (hms-website/ using JSP + Tomcat)

```

swift
CopyEdit
hms-website/
└─ src/
    └─ main/
        └─ java/com/hms/web/
            ├── HomeController.java
            ├── FeedbackController.java
            └─ ReservationController.java
        └─ resources/
        └─ webapp/
        └─ WEB-INF/

```

```
|
|
|
|   └─ views/
|       ├── index.jsp
|       ├── about.jsp
|       ├── gallery.jsp
|       ├── feedback.jsp
|       └─ reserve.jsp
|   └─ static/
|       ├── css/
|       ├── js/
|       └─ images/
└─ pom.xml
```

4. React Native Mobile App (hms-mobile/ – Expo or Bare)

```
css
CopyEdit
hms-mobile/
├─ src/
│   ├── screens/
│   │   ├── LoginScreen.js
│   │   ├── DashboardScreen.js
│   │   ├── BookingScreen.js
│   │   ├── StaffPanel.js
│   │   └─ FeedbackScreen.js
│   ├── components/
│   │   ├── RoomCard.js
│   │   └─ CustomButton.js
│   ├── navigation/
│   │   └─ AppNavigator.js
│   └─ assets/
```

```

|   |   |   | fonts/
|   |   |   | icons/
|   |   |   | images/
|   |   |   |
|   |   |   | services/
|   |   |   |   | api.js
|   |   |   |   | auth.js
|   |   |   |
|   |   |   | context/
|   |   |   |   | AuthContext.js
|   |   |   |
|   |   |   | App.js
|   |   |   |
|   |   |   | app.json

```

HMS Backend – Full File Structure & Database Schema

Backend Structure Overview

(Already described above, refer to previous structure section)

Database Schema – Core Modules

The HMS project uses **PostgreSQL** with normalized relational tables. Below is a detailed schema per module.

1. Users & Roles

```

users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(150) UNIQUE,
  password TEXT,
  mobile VARCHAR(20),
  role VARCHAR(50), -- OWNER, MANAGER, STAFF, CLEANER, CHEF
  hotel_id INT REFERENCES hotels(id),
  is_active BOOLEAN DEFAULT TRUE,

```

```
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
)  
  
roles (  
    id SERIAL PRIMARY KEY,  
    role_name VARCHAR(50) UNIQUE  
)
```

2. 🏨 Hotels

```
hotels (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(200),  
    address TEXT,  
    email VARCHAR(150),  
    phone VARCHAR(20),  
    gst_number VARCHAR(50),  
    created_at TIMESTAMP  
)
```

3. 🛏 Rooms

```
rooms (  
    id SERIAL PRIMARY KEY,  
    hotel_id INT REFERENCES hotels(id),  
    room_number VARCHAR(10),  
    floor INT,  
    type VARCHAR(50), -- AC/NON-AC  
    beds INT,  
    price DECIMAL(10,2),  
    status VARCHAR(20), -- EMPTY, OCCUPIED, DIRTY, CLEANING  
    created_at TIMESTAMP  
)
```

4. 📅 Bookings

```
bookings (  
  id SERIAL PRIMARY KEY,  
  guest_id INT REFERENCES guests(id),  
  room_id INT REFERENCES rooms(id),  
  user_id INT REFERENCES users(id),  
  checkin TIMESTAMP,  
  checkout TIMESTAMP,  
  status VARCHAR(20), -- BOOKED, COMPLETED, CANCELLED  
  total_amount DECIMAL(10,2),  
  created_at TIMESTAMP  
)
```

5. 👤 Guests

```
guests (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  mobile VARCHAR(20),  
  email VARCHAR(150),  
  id_proof_type VARCHAR(50),  
  id_number VARCHAR(100),  
  photo_url TEXT,  
  created_at TIMESTAMP  
)
```

6. 📄 Invoices

```
invoices (  
  id SERIAL PRIMARY KEY,  
  booking_id INT REFERENCES bookings(id),  
  invoice_number VARCHAR(50) UNIQUE,  
  issue_date TIMESTAMP,  
  total_amount DECIMAL(10,2),  
  gst_amount DECIMAL(10,2),  
  discount_amount DECIMAL(10,2),
```

```
    payment_status VARCHAR(20), -- PAID, PARTIAL, PENDING
    generated_by INT REFERENCES users(id)
)
```

7. Inventory

```
inventory (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    quantity INT,
    unit VARCHAR(20),
    threshold INT,
    location TEXT,
    last_used TIMESTAMP,
    hotel_id INT REFERENCES hotels(id)
)
```

8. Housekeeping

```
housekeeping_tasks (
    id SERIAL PRIMARY KEY,
    room_id INT REFERENCES rooms(id),
    assigned_to INT REFERENCES users(id),
    status VARCHAR(20), -- PENDING, COMPLETED
    task_date DATE,
    remarks TEXT
)
```

9. Staff Attendance & Leaves

```
attendance (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES users(id),
    checkin TIMESTAMP,
    checkout TIMESTAMP,
    method VARCHAR(20), -- MANUAL, BIOMETRIC
)
```

```

    status VARCHAR(20) -- PRESENT, ABSENT
)

leave_requests (
    id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES users(id),
    from_date DATE,
    to_date DATE,
    reason TEXT,
    status VARCHAR(20), -- PENDING, APPROVED, REJECTED
    applied_at TIMESTAMP
)

```

10. Feedback / Service Requests

```

feedback (
    id SERIAL PRIMARY KEY,
    guest_id INT REFERENCES guests(id),
    hotel_id INT REFERENCES hotels(id),
    type VARCHAR(20), -- COMPLAINT, REQUEST, FEEDBACK
    content TEXT,
    status VARCHAR(20), -- PENDING, RESOLVED
    submitted_at TIMESTAMP
)

```

11. Finance

```

expenses (
    id SERIAL PRIMARY KEY,
    hotel_id INT REFERENCES hotels(id),
    department VARCHAR(50),
    description TEXT,
    amount DECIMAL(10,2),
    expense_date DATE
)

```

```

revenues (

```

```
    id SERIAL PRIMARY KEY,  
    hotel_id INT REFERENCES hotels(id),  
    source VARCHAR(50),  
    amount DECIMAL(10,2),  
    received_date DATE  
)
```

12. 📺 IoT & QR Access

```
device_registry (  
    id SERIAL PRIMARY KEY,  
    room_id INT REFERENCES rooms(id),  
    device_id VARCHAR(100),  
    device_type VARCHAR(20), -- LOCK, AC, LIGHT  
    is_active BOOLEAN,  
    last_heartbeat TIMESTAMP  
)
```

```
qr_access (  
    id SERIAL PRIMARY KEY,  
    room_id INT REFERENCES rooms(id),  
    guest_id INT REFERENCES guests(id),  
    token TEXT,  
    valid_from TIMESTAMP,  
    valid_until TIMESTAMP,  
    scanned BOOLEAN  
)
```

13. ⚙️ Global Settings

```
global_settings (  
    id SERIAL PRIMARY KEY,  
    hotel_id INT REFERENCES hotels(id),  
    key VARCHAR(100),  
    value TEXT,  
    updated_at TIMESTAMP  
)
```

)

Let me know if you'd like this schema visualized as an **ERD (Entity Relationship Diagram)** or exported to PDF, SQL, or CSV.

Hotel Management System – Low-Level Design (LLD)

This document details the **Low-Level Design (LLD)** for the HMS project, covering modules, classes, key interfaces, database interactions, request flows, service layers, and helper utilities.

Architectural Layers

1. Presentation Layer (Frontend)

- JavaFX Desktop: Staff/Manager operations
- JSP Website: Guest/Customer access
- React Native App: Mobile interface for staff, owner

2. Service Layer (Business Logic)

- Booking logic, availability check, price computation
- Attendance calculations, shift validation
- Device access control, QR handling, OTP verification

3. Persistence Layer (Repository/DAO)

- Spring Data JPA Repositories for all entities
- Query methods with pagination and custom filters

4. Security Layer

- Spring Security + JWT filter
- Role-based access (RBAC)
- Session, login/logout tokens, expiry handling

Key Components by Module

Authentication Module

- **Entities:** User, Role
- **Controllers:** AuthController
- **Services:** AuthService, JwtUtil
- **DTOs:** LoginRequest, RegisterRequest, ProfileDTO
- **Logic:**
 - Encrypt passwords using BCrypt
 - Generate and verify JWT tokens
 - Role validation for endpoints

Room Module

- **Entities:** Room
- **Controllers:** RoomController
- **Services:** RoomService
- **Logic:**
 - CRUD rooms
 - Filter by type, floor, status
 - Validate uniqueness (room no./hotel)



Booking Module

- **Entities:** Booking, Guest
- **Controllers:** BookingController
- **Services:** BookingService
- **Logic:**
 - Check room availability by date range
 - Calculate total cost: days × price × taxes
 - Assign room and update status



Billing Module

- **Entities:** Invoice
- **Controllers:** BillingController
- **Services:** InvoiceService
- **Logic:**
 - Generate invoice after booking or checkout
 - Add services & tax calculation
 - Export invoice as PDF (iText)



Housekeeping Module

- **Entities:** HousekeepingTask
- **Controllers:** HousekeepingController
- **Services:** HousekeepingService
- **Logic:**
 - Assign task to cleaner
 - Track cleaning status
 - Auto-update room cleanliness



Staff Module

- **Entities:** Staff (extending User), Attendance, LeaveRequest
- **Controllers:** StaffController
- **Services:** AttendanceService, LeaveService
- **Logic:**
 - Manual/Biometric check-in

- Leave approvals
- Attendance calendar logic

Inventory Module

- **Entities:** InventoryItem
- **Controllers:** InventoryController
- **Services:** InventoryService
- **Logic:**
 - Alert on low quantity
 - Add usage logs

Finance Module

- **Entities:** Expense, Revenue
- **Controllers:** FinanceController
- **Logic:**
 - Monthly breakdowns
 - Department-wise categorization
 - Salary tracking support

IoT Module

- **Entities:** DeviceRegistry, QRAccess
- **Controllers:** IoTController, QRController
- **Logic:**
 - Generate secure QR
 - Validate QR on scan
 - Trigger smart lock/AC/light control

Feedback Module

- **Entities:** Feedback
- **Controllers:** FeedbackController
- **Logic:**
 - Guest submits complaint or service request
 - Staff responds & closes



Report Module

- **Controllers:** ReportController
- **Services:** PdfExportService, ExcelExportService
- **Logic:**
 - Export filters applied
 - Templates loaded and filled



Entity Relationship Mapping (Simplified)

- One Hotel → Many Rooms, Bookings, Staff
- One Booking → One Room, One Guest
- One Invoice → One Booking
- One User (staff) → Many Attendances, Tasks
- One Room → Many HousekeepingTasks, QR Access



Core Utility Classes

- **JwtUtil** – Generate, validate tokens
- **EmailUtil** – SMTP alerts (optional)
- **DateUtil** – Parse/format timestamps
- **PdfGenerator** – iText-based export
- **FileUtil** – Save images, ID cards



Testing & Validation

- Unit Tests (JUnit 5)
- Integration Tests using @SpringBootTest
- Swagger docs for all APIs
- Postman collections for functional tests